

The modern data stack

How data gets from where it is born to where we analyze it

```
python · pipeline.py  
  
>>> raw = extract("statcan")  
>>> tidy = transform(raw)  
>>> load(tidy, "warehouse")  
>>> analyze(tidy) # finally, the fun part
```



Agenda

1

The data lifecycle

Where data comes from, where it is kept, and when it should go away

~15 min

2

Organizing and managing data

Tidy tables, keys, schemas, file formats, metadata, quality checks and governance

~20 min

3

Analytical pipelines

ETL and ELT, orchestration, lineage, and the tools used at each layer

~20 min

4

Putting it together

A worked case, use cases, where AI fits in, and a design exercise

~15 min

BY THE END OF CLASS

You should be able to take a vague data request, for example "how did our sales react to the price change?", and sketch the pipeline that answers it.

We will also name the tool we would use at each step.

In most projects, the model is the last and shortest step

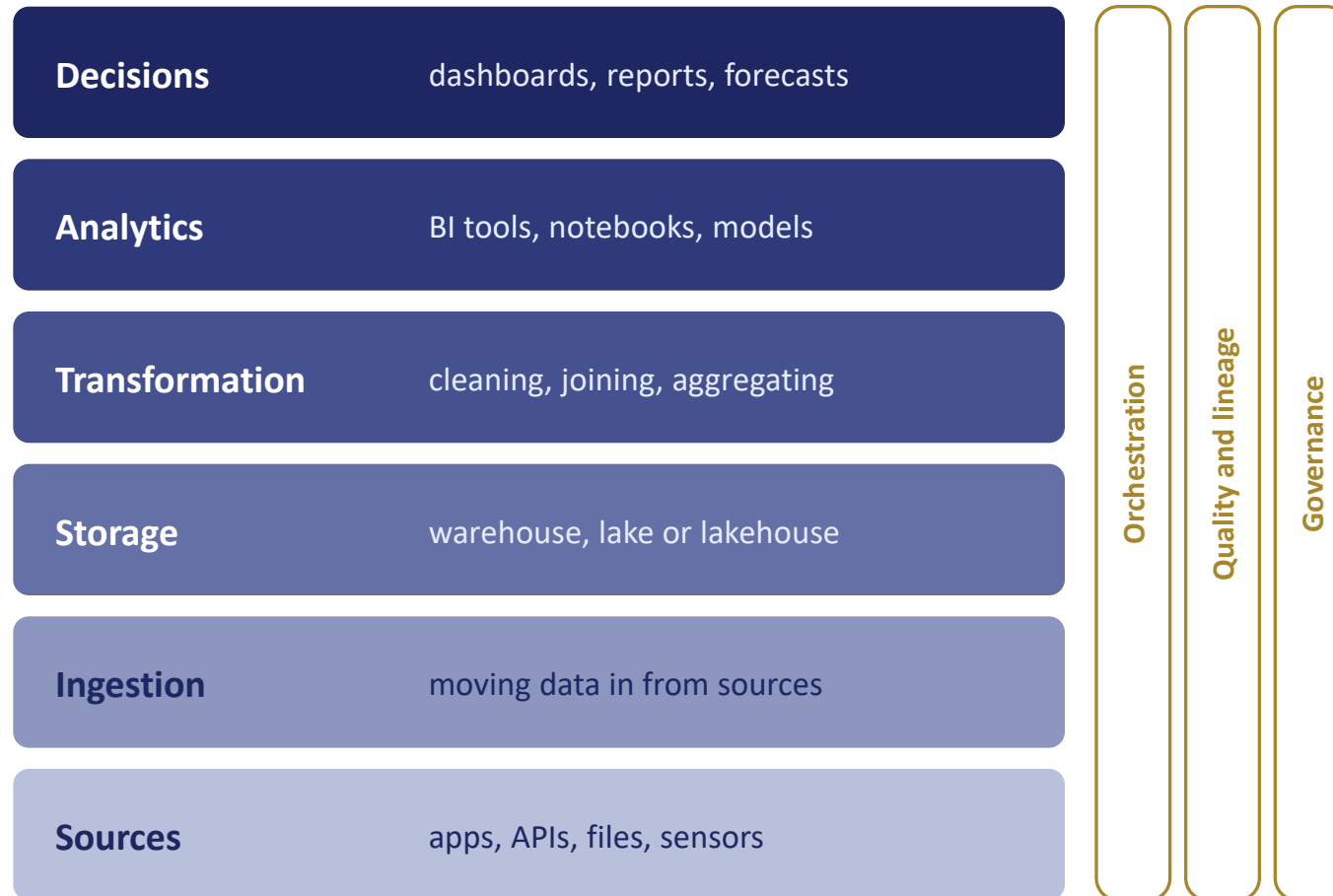
Suppose a coffee chain asks us whether its September price change hurt weekend sales. The regression itself takes an afternoon. Getting the data into shape to run it can take weeks.



Steps 1 to 4 are the subject of today. Reports usually describe them in a paragraph.

What gets published

A data stack is a set of layers from source to decision



Data flows up, from bottom to top

What makes it "modern"?

- It runs in the cloud, so storage and computing are **rented by the hour** instead of bought up front
- Each layer is a **separate tool** that does one job, and tools can be swapped without rebuilding everything
- Most transformation happens **inside the warehouse** in SQL (this is ELT, more on it later)
- Much of it is open-source or freely-available, so you can practice at home



PART 1 OF 4

The data lifecycle

Where data comes from, where it is kept, and when it should go away

Sources

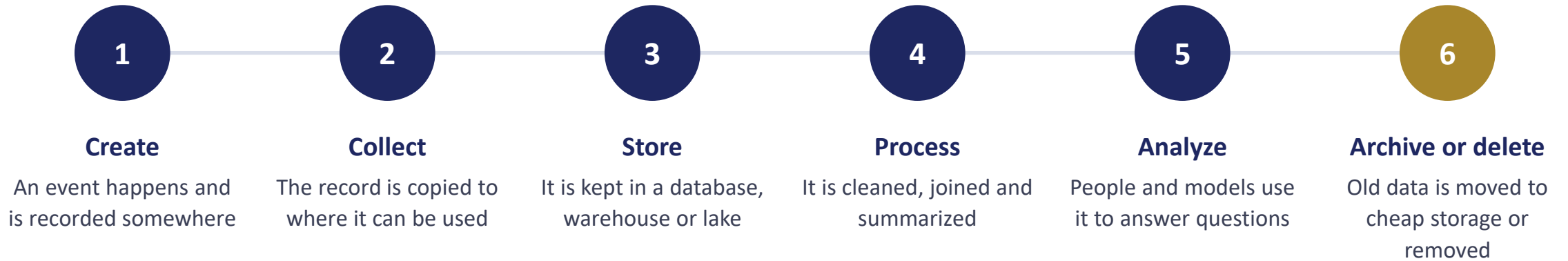
Batch and streaming

Storage

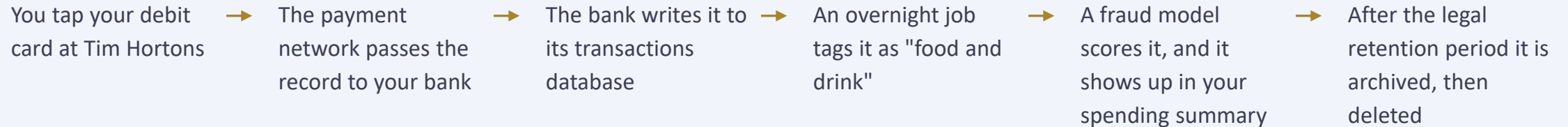
OLTP and OLAP

Retention

Data goes through six stages, from creation to deletion



FOR EXAMPLE, ONE COFFEE PURCHASE



Data is born in many places, and in different shapes

Structured

Rows and columns, each column with a fixed type. Easy to query with SQL.

```
id,date,amount
1041,2026-09-14,4.85
1042,2026-09-14,12.10
```

EXAMPLES

Bank transactions, grades, StatCan tables

Semi-structured

Has labels and nesting, but no fixed table shape. Typical of APIs and logs.

```
{"id": 1041,
 "items": ["latte"],
 "store": {"city":
  "St. Catharines"}}
```

EXAMPLES

API responses, web logs, app events

Unstructured

No predefined structure at all. Needs extra work before we can count anything.

```
"Latte was great, but
the line was way
too long, and the
patio was closed..."
```

EXAMPLES

News text, PDFs, images, audio

Where it comes from: company databases, web and mobile apps, public APIs (Statistics Canada, FRED, the Bank of Canada Valet API), spreadsheets, sensors, and web scraping.

COLLECT

Batch or streaming: it depends on how fresh answers must be

Batch

Collect for a while, then process everything in one go.

- Simpler and cheaper to run
- Easy to re-run if something breaks
- Answers are only as fresh as the last run

Tools: Python on a schedule, Airflow, dbt

Streaming

Process each event within seconds of it happening.

- Answers in seconds, or less
- Harder to build and debug, and it costs more, since it runs all the time

Tools: Apache Kafka, Apache Flink, Spark Streaming

HOW OLD CAN THE ANSWER BE?



Rule of thumb: use batch, unless someone loses money while waiting for the answer.

Databases, warehouses, lakes and lakehouses do different jobs

	What it holds	Built for	Main users	Examples
Database	Current records of a running app	Many small reads and writes	Software developers	PostgreSQL, MySQL, SQLite
Data warehouse	Cleaned, structured history	Large analytical SQL queries	Analysts	Snowflake, BigQuery, Redshift
Data lake	Raw files of any kind	Storing everything cheaply	Data engineers and scientists	Amazon S3, Azure Data Lake
Lakehouse	Raw files with tables on top	Both of the above in one place	In principle, everyone	Databricks, Apache Iceberg

A rough analogy: the lake is your Downloads folder, where everything lands as-is. The warehouse is the neatly-labelled filing cabinet. Analysts (and, arguably, most of you after graduation) spend their time in the warehouse.

OLTP systems record events, OLAP systems answer questions

OLTP: online transaction processing

Stores data row by row. Fast when we need one whole record, for example to show a customer their order.

order_id	date	customer	amount
1041	09-14	A. Chen	4.85
1042	09-14	R. Patel	12.10
1043	09-15	M. Roy	7.40
1044	09-15	J. Smith	3.25

Reads one row

```
SELECT * FROM orders
WHERE order_id = 1042;
```

OLAP: online analytical processing

Stores data column by column. Fast when we need a few columns across millions of rows.

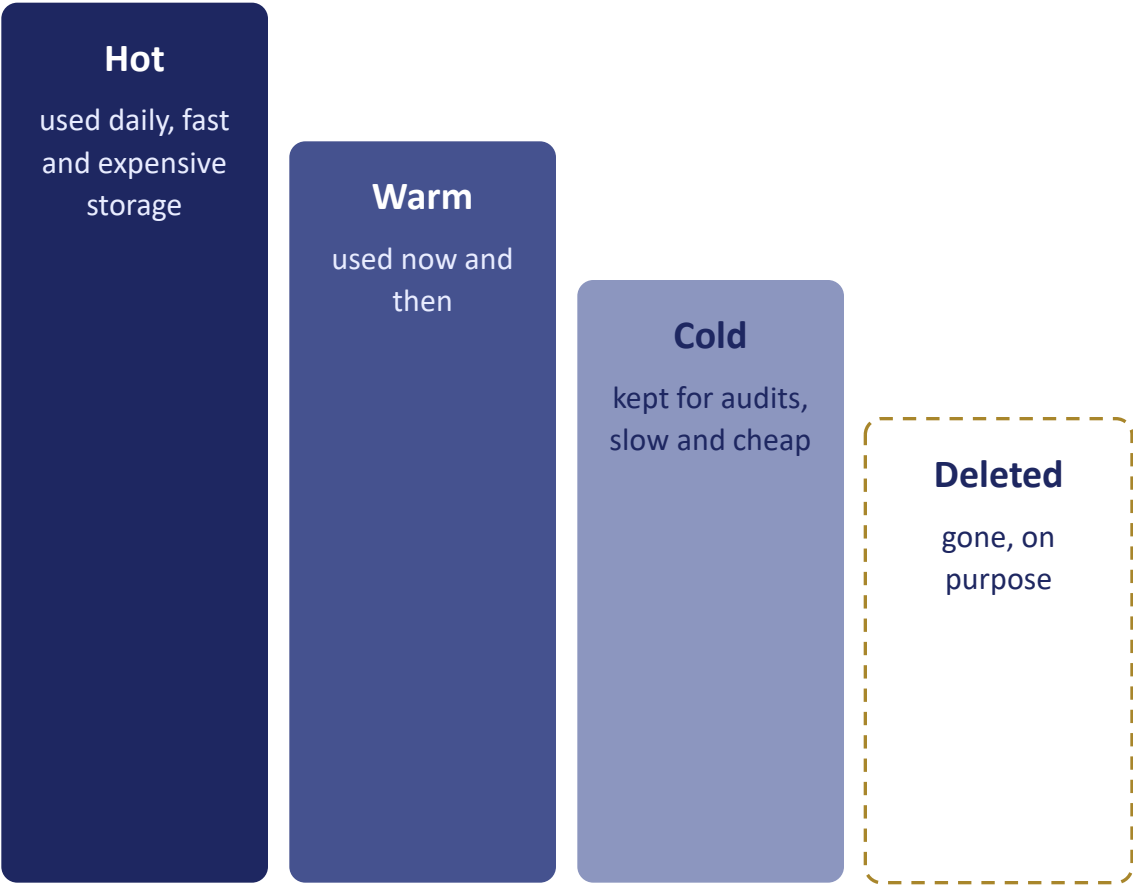
order_id	date	customer	amount
1041	09-14	A. Chen	4.85
1042	09-14	R. Patel	12.10
1043	09-15	M. Roy	7.40
1044	09-15	J. Smith	3.25

Reads two columns, skips the rest

```
SELECT date, SUM(amount)
FROM orders GROUP BY date;
```

Your bank app runs on OLTP. The bank's monthly report on spending patterns runs on OLAP, usually a warehouse.

Data should have an end date



Older data moves to cheaper storage as it gets used less

Why not keep everything forever?

- Old records can **leak in a breach** just as easily as new ones, and old personal data is rarely worth the risk
- In Canada, PIPEDA says personal information should be kept **only as long as needed** for the purpose it was collected for

AN ILLUSTRATIVE RETENTION SCHEDULE

Data	Kept for
Raw website logs	90 days
Customer transactions	7 years
Monthly aggregates	Indefinitely



PART 2 OF 4

Organizing and managing data

Habits that make data easy to find, trust and reuse

Tidy data

Keys and schemas

Naming

File formats

Metadata

Quality

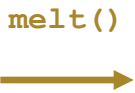
Governance

Tidy data: one variable per column, one observation per row

Wickham (2014) gives three rules: each variable is a column, each observation is a row, and each type of observational unit gets its own table. In other words, the same numbers can be stored tidy or not.

Wide (messy): years hide in column names

province	2024	2025
ON	7.0	7.8
QC	5.3	5.9
BC	5.6	6.2



Long (tidy): year is now a variable of its own

province	year	rate
ON	2024	7.0
ON	2025	7.8
QC	2024	5.3
QC	2025	5.9
BC	2024	5.6
BC	2025	6.2

```
tidy = wide.melt(  
  id_vars="province",  
  var_name="year",  
  value_name="rate")
```

Tidy data is longer and looks less friendly. Still, pandas, SQL and most plotting libraries expect it.

Keys let us join tables without copying data

customers

customer_id (PK)	name	city
C01	A. Chen	St. Catharines
C02	R. Patel	Welland

orders

order_id (PK)	customer_id (FK)	amount
1041	C01	4.85
1042	C02	12.10
1043	C01	7.40

A **primary key** (PK) identifies each row. A **foreign key** (FK) points to a primary key in another table.

```
sql · join.sql

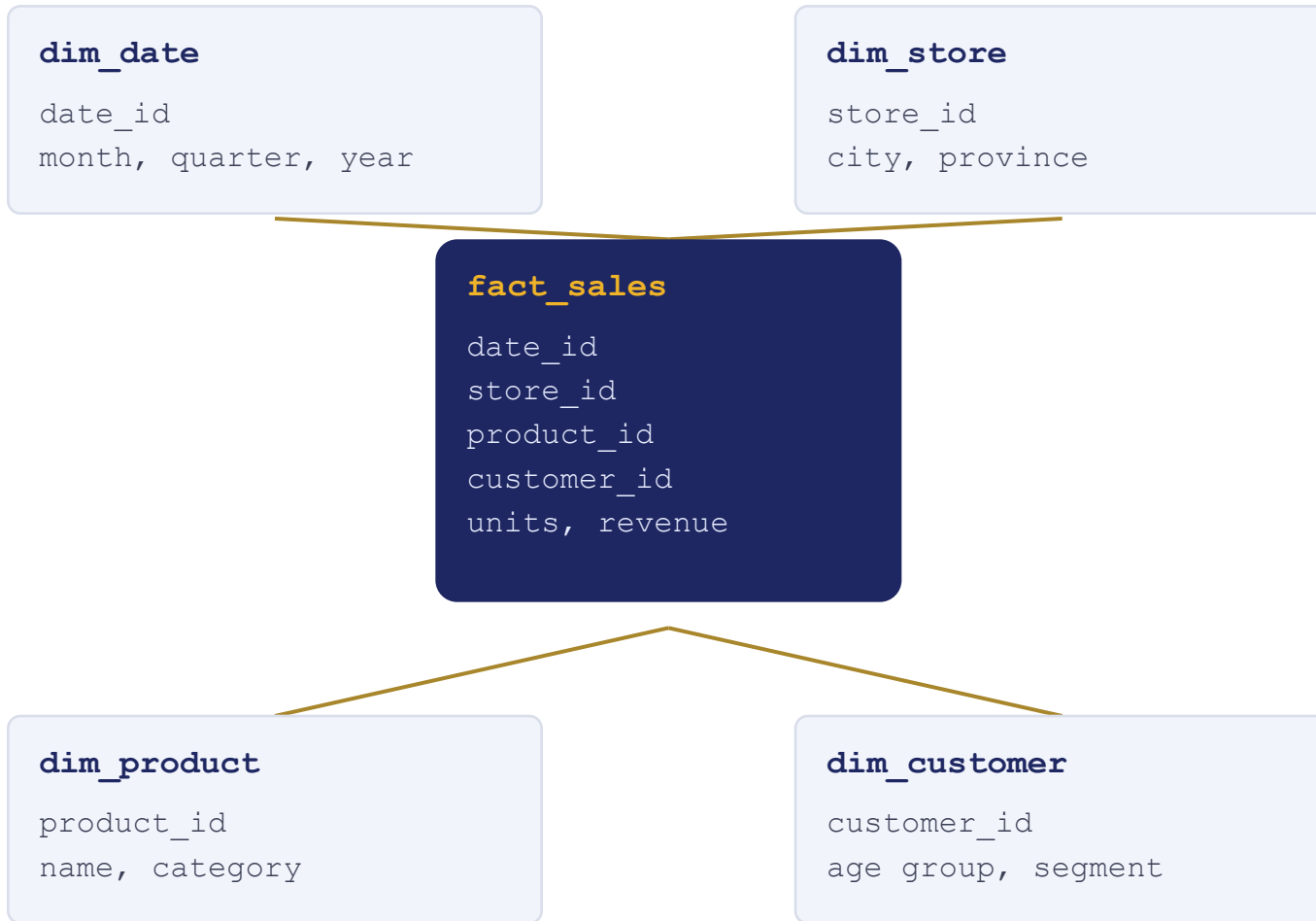
SELECT o.order_id, c.city, o.amount
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id;
```

Why not keep the city in the orders table?

If A. Chen moves to Welland, we change **one row** in customers. With the city copied into every order, we would need to find and fix all of them, and sooner or later we would miss one.

Storing each fact once is called **normalization**.

Star schemas put facts in the middle and context around them



Two kinds of tables

The **fact table** has one row per event, here one row per sale. It holds keys and numbers, and it gets very long.

Dimension tables describe the context of each sale, for example the store and the date. They are what we filter and group by.

FOR EXAMPLE

"Revenue by province and quarter" is the fact table joined to `dim_store` and `dim_date`, then summed. Most dashboards are built on queries like this one.

Good folder and file names make a project easy to re-run

```
bash · tree coffee-sales-project
coffee-sales-project/
├── data/
│   ├── raw/          # read-only
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_explore_sales.ipynb
│   └── 02_price_effect.ipynb
├── src/
│   └── build_weekly_sales.py
├── reports/
└── README.md
```

- Treat raw data as **read-only**. Every change happens in code, so anyone can repeat it from the start
- Use snake_case and no spaces: cpi_monthly.csv, and not "CPI Monthly (new).csv"
- Write dates as YYYY-MM-DD, so that files sort in the right order: 2026-10-06_sales.csv
- Number your notebooks in the order they should be run
- Keep a README that says what each folder holds and where the raw data came from
- Let GitHub keep the old versions. No more sales_FINAL_v2_fixed.csv

For large analytical tables, Parquet beats CSV

Format	Open in a text editor?	Keeps column types?	Good for
CSV	Yes	No, everything is text	Small tables, sharing with anyone
Excel (.xlsx)	No	Partly	Tables people will edit by hand
JSON	Yes	Partly	API responses, nested records
Parquet	No	Yes	Large analytical tables

Why analysts like Parquet

- It is stored by column, like the OLAP tables from Part 1, so reading two columns of a wide table is fast
- It is highly-compressed, and often several times smaller than the same data saved as CSV

```
python · formats.py

df.to_csv("returns.csv")
df.to_parquet("returns.parquet")

# read back only what we need
pd.read_parquet("returns.parquet",
               columns=["date", "ret"])
```

Metadata is data about the data

A data dictionary lists every column with its type, unit and meaning. For example, for a table of monthly provincial CPI pulled from Statistics Canada:

Column	Type	Unit	Meaning
ref_date	date	month	Reference month, stored as the first day
geo	text		Province name, as StatCan spells it
cpi	float	index, 2002 = 100	All-items consumer price index
pulled_at	timestamp		When our script downloaded the data

Without the unit row, is cpi a level or a percent change? Without pulled_at, which StatCan revision are we looking at? Two lines of metadata answer both.

In a company, metadata lives in a **data catalogue**, which is a searchable list of every table, its owner and its columns. Common tools are DataHub, Atlan and the documentation built into dbt.

Source of the example table: Statistics Canada, Table 18-10-0004-01, Consumer Price Index, monthly, not seasonally adjusted.

Data quality can be tested, the same way code is tested

Check	The question	One line of pandas
Completeness	Are any values missing?	<code>assert df["ret"].notna().all()</code>
Uniqueness	Is any order counted twice?	<code>assert not df["order_id"].duplicated().any()</code>
Validity	Are values in a sensible range?	<code>assert df["rate"].between(0, 100).all()</code>
Consistency	Does every order have a customer?	<code>assert orders["cust_id"].isin(customers["cust_id"]).all()</code>
Timeliness	Is the latest month really there?	<code>assert df["date"].max() >= "2026-09-01"</code>

In a company, checks like these run automatically every time the pipeline runs, using tools such as dbt tests, Great Expectations or Soda. If a check fails, the pipeline stops and someone gets an email. In other words, we would rather have the pipeline stop than have it quietly produce wrong numbers.

Governance decides who can see what, and why

WHO GETS ACCESS TO WHAT: AN EXAMPLE

	Monthly sales totals	Customer emails	Card numbers
Analyst	Read	Masked	No access
Data engineer	Read	Read	Masked
Marketing intern	Read	No access	No access
Fraud team	Read	Read	Read

Masking means the value is replaced, for example j.smith@brocku.ca becomes j*****@brocku.ca, or a random customer ID.

Common rules

- People get access to the data their job needs, and nothing more. This is called **least privilege**
- Personal information is masked or swapped for an ID before most analysts see it
- Every table has an owner, who answers questions about it and approves access
- Access is logged, so that later we can tell who looked at what



PART 3 OF 4

Analytical pipelines

How raw data becomes a number on a dashboard, on schedule and with a paper trail

ETL and ELT

Extract

Transform

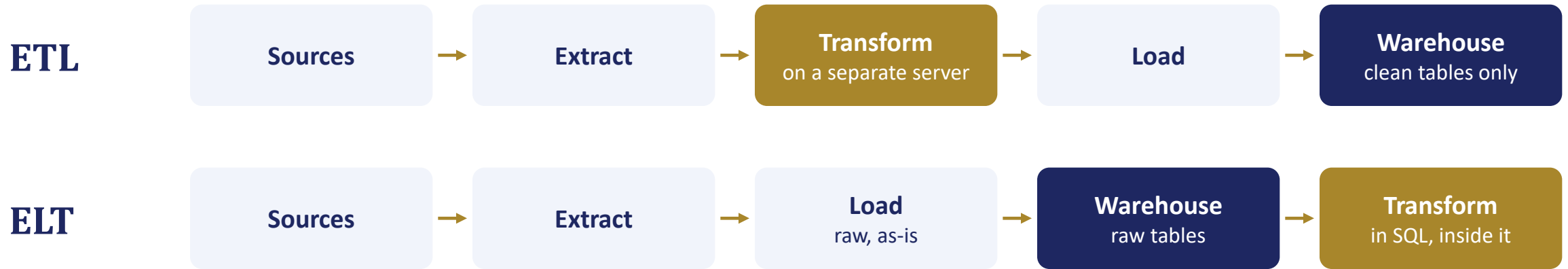
Load

Orchestration

Lineage

Tools

ETL transforms before loading, ELT transforms after



Why most new pipelines are ELT

- Cloud warehouses made storage cheap and SQL fast, so it became easier to load first and clean later
- Further, the raw data stays in the warehouse. If the cleaning logic changes, we re-run the SQL, and there is no need to extract again

When ETL still makes sense

- Personal data must be masked before it lands anywhere, for privacy reasons
- The source is huge and we only need a small piece of it, for example one ticker out of millions of trades

Extract: pulling the USD/CAD rate from the Bank of Canada

```
python · extract_fx.py

import requests, pandas as pd

url = "https://www.bankofcanada.ca/valet/"
    "observations/FXUSDCAD/json"
r = requests.get(url, params={"recent": 30})
r.raise_for_status() # stop if the request failed

obs = r.json()["observations"]
df = pd.DataFrame({
    "date": [o["d"] for o in obs],
    "usdcad": [float(o["FXUSDCAD"]["v"]) for o in obs]})

df.to_parquet("data/raw/usdcad_2026-10-06.parquet")
```

What is going on here

- The API sends back nested JSON, which we flatten into a two-column table
- We save the raw pull, with the date in the file name, before changing anything
- Here we take the last 30 days. Taking only what is new since the last run is called an **incremental** extract
- If the API needs a key, the key goes in an environment variable, and never in the code on GitHub

At company scale, tools such as Fivetran and Airbyte offer ready-made connectors for hundreds of sources.

Transform: turning raw orders into monthly revenue

```
sql · models/monthly_revenue.sql

WITH clean_orders AS (
  SELECT DISTINCT order_id, customer_id,
    CAST(order_date AS DATE) AS order_date,
    amount
  FROM raw.orders
  WHERE amount > 0 -- drop refunds, test rows
)
SELECT DATE_TRUNC('month', order_date) AS month,
  c.city,
  SUM(o.amount) AS revenue
FROM clean_orders o
JOIN customers c USING (customer_id)
GROUP BY 1, 2;
```

What transformation usually means

- Fixing types, missing values and duplicates
- Making units and labels agree, so "Ont." and "Ontario" both become ON
- Joining tables on their keys
- Building new variables, for example returns from prices
- Aggregating to the level we analyze at, here city and month

dbt lets us keep each step as one SELECT in its own file. It then works out the order to run them in. In pandas, the same thing is a `merge()` and a `groupby()`.

Load: there are three ways to write data into a warehouse

WAYS TO LOAD A TABLE

Full refresh

Delete the table and write it again. Simple, but slow for big tables.

Append

Add only the new rows. Fast, but a re-run can create duplicates.

Upsert (merge)

Update rows that already exist, insert the rest. Safe to re-run, a bit more work to write.

WHERE THE DATA LANDS

Snowflake

Cloud warehouse, pay for computing time used

Google BigQuery

Cloud warehouse, pay per amount of data scanned

Amazon Redshift

Cloud warehouse inside Amazon Web Services

Databricks

Lakehouse built on Apache Spark

PostgreSQL

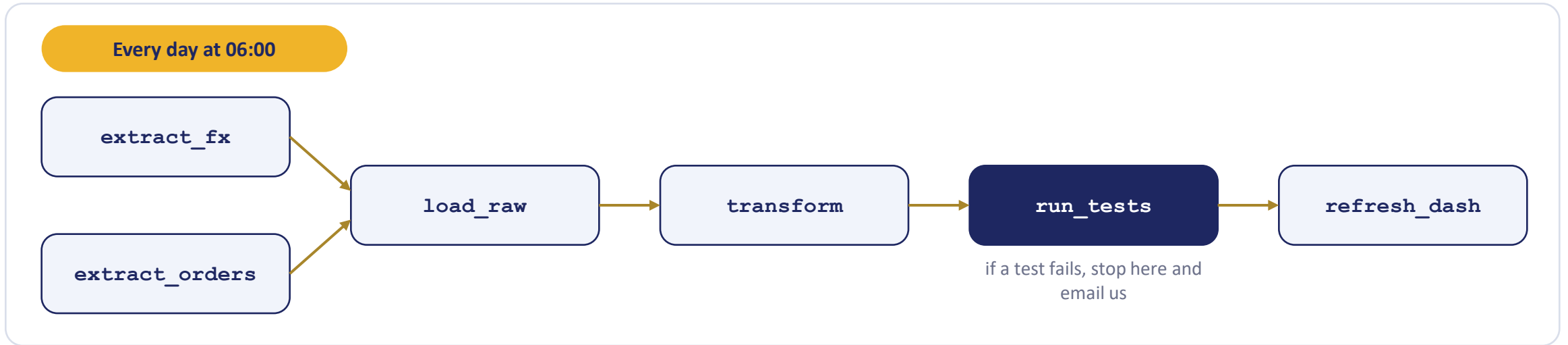
Free, open-source database that works for small warehouses too

DuckDB

Free, runs on your laptop or Codespace, reads Parquet directly

For practice, DuckDB is the easiest place to start: free, and no account needed.

Orchestration runs each step on schedule, in the right order



A pipeline is a DAG

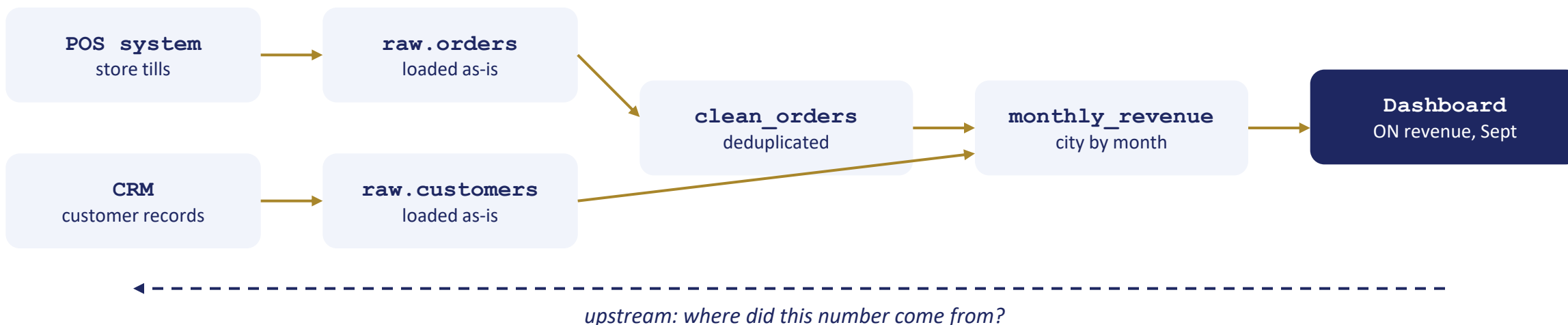
DAG stands for directed acyclic graph. In other words, every arrow points forward and nothing loops back, so there is always a correct order to run things in. Here both extracts can run at the same time, but transform has to wait for load_raw.

What the orchestrator does

- Starts the run on schedule, and retries a step if it fails
- Keeps a log of every run, so we can see what broke and when

Tools: Apache Airflow, Dagster, Prefect. For small jobs, GitHub Actions on a schedule is often enough.

Lineage traces any number back to where it came from



Looking upstream

The Ontario number on the dashboard looks too low. Lineage tells us exactly which tables and steps fed into it, so we check those first, and do not search the whole warehouse.

Looking downstream

The IT team wants to rename a column in raw.orders. Lineage shows every table and dashboard that would break, before anyone presses the button.

Tools: dbt builds a lineage graph automatically from your SQL. OpenLineage, DataHub and Unity Catalog do it across a whole company.

The modern data stack, one layer at a time

Layer	Job	Used in industry	Free, for practice
Ingestion	Move data in from sources	Fivetran, Airbyte, Kafka	Python with requests
Storage	Keep it, query it	Snowflake, BigQuery, Databricks	DuckDB, PostgreSQL
Transformation	Clean, join, aggregate	dbt, Spark	dbt Core, pandas
Orchestration	Run steps in order, on time	Airflow, Dagster, Prefect	GitHub Actions
Quality	Catch bad data early	Great Expectations, Soda	pandas asserts, dbt tests
Catalogue and lineage	Know what exists and where it came from	DataHub, Atlan, Unity Catalog	dbt docs
BI and apps	Show results to people	Tableau, Power BI, Looker	Streamlit, Plotly Dash

Tool names change every few years, while the layers change much more slowly. For this reason, we focus on the layers.

Analytics, at the top, answers four kinds of questions

DESCRIPTIVE

What happened?

"Ontario revenue fell in September."

Tools: SQL, dashboards

DIAGNOSTIC

Why did it happen?

"Weekend visits dropped after the price change."

Tools: Drill-downs, regression

PREDICTIVE

What will happen?

"October revenue will likely be lower still."

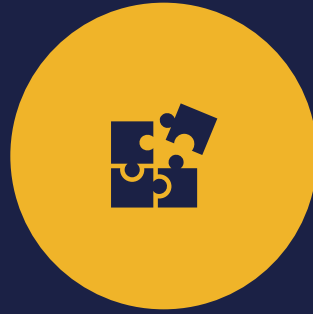
Tools: Time-series models, machine learning

PRESCRIPTIVE

What should we do?

"Run a weekend promotion in these 12 stores."

Tools: Optimization, experiments



PART 4 OF 4

Putting it together

Worked cases, where AI fits in, and a pipeline of your own

A worked case

Use cases

AI in the stack

Checklist

Exercise

WORKED CASE

The price question from the agenda, one step at a time

Step	What we do	Tools
Collect	Pull daily orders from the till system and prices from the product list	Python, Airbyte
Store	Load the raw tables into the warehouse as-is, with the date of the pull	DuckDB, Snowflake
Transform and test	Clean orders, join them to stores and dates, sum to store by week, and test for gaps	dbt, SQL, dbt tests
Analyze	Compare weekend sales before and after the change, against stores that kept the old price	Notebook, regression
Share	A weekly dashboard, with lineage back to the raw orders	Power BI, Streamlit

Interestingly, the regression is one row out of five. Most of the time goes into the rows above it, and this is also where most things can go wrong.

Comparing with stores that kept the old price lets us take into the account things like weather and holidays, which affect every store.

Where you will meet this stack after graduation

Card fraud detection

A bank scores every card payment before approving it. Suspicious ones are held for review.

Streaming, milliseconds

TYPICAL STACK

Kafka, a scoring model, alerts

Retail inventory

A grocery chain forecasts tomorrow's demand for each product in each store, and orders stock.

Batch, nightly

TYPICAL STACK

Warehouse, dbt, forecasting model

Nowcasting GDP

A central bank estimates this quarter's GDP from monthly data, weeks before the official release.

Batch, monthly

TYPICAL STACK

Public APIs, time-series model

Marketing analytics

An online store tracks which ads lead to purchases, and moves budget toward the ones that work.

Batch, hourly or daily

TYPICAL STACK

Event tracking, warehouse, BI

Most business pipelines are batch. Streaming shows up where a delay costs money right away.

AI tools now show up at almost every layer of the stack

Layer	What AI tools do well	What we still have to check
Ingestion	Turn PDFs, emails and news text into tables	Values that are not actually in the document
Transformation	Write SQL and pandas code from a plain description	Code that runs fine but answers a different question
Quality and catalogue	Suggest tests, write column descriptions	Descriptions that sound right but are wrong
Analytics	Answer questions about the warehouse in plain English	Wrong joins behind a confident answer

THE COURSE RULE STILL APPLIES

If you cannot explain what the generated query does, row by row, do not put its output on a dashboard. Lineage and tests from earlier in this lecture are how we check AI output, the same way we check our own.

Eight checks before you share any analysis



Raw data is saved with a date, and never edited

Part 3



Each table comes with a short data dictionary

Part 2



Every step lives in code, in a GitHub repo

Part 2



Quality tests pass before results are shared

Part 2



Tables are tidy, and each has a clear key

Part 2



Personal data is masked, or better, never collected

Part 2



Large tables are stored as Parquet, not CSV

Part 2



Any number can be traced back to its source

Part 3

If a classmate could not re-run your project from the repo alone, something on this list is missing.

Glossary

Term	Meaning
Pipeline	The steps that move data from sources to where it is used
ETL, ELT	Extract, transform, load; ELT loads before it transforms
Warehouse	A database built for analytical queries on clean history
Data lake	Cheap storage for raw files of any type
Lakehouse	A data lake with tables and SQL on top
OLTP, OLAP	Systems that record transactions, and systems that analyze them
Schema	The structure of a table: its columns and their types

Term	Meaning
Primary key	A column that identifies each row of a table
Foreign key	A column that points to a primary key in another table
Star schema	A fact table with dimension tables around it
Orchestration	Running pipeline steps in order and on a schedule
DAG	Directed acyclic graph: arrows only point forward
Lineage	The record of where each piece of data came from
Idempotent	A step that gives the same result however many times it runs

YOUR TURN

Design a pipeline for a Niagara tourism dashboard

The scenario: a regional tourism office wants a weekly dashboard comparing hotel occupancy in Niagara with the USD/CAD exchange rate. Do U.S. visitors come more often when the Canadian dollar is cheap?

1

Which sources do you need, and how would you extract from each?

2

Batch or streaming? How often should it run?

3

Where is the data stored, and in what format?

4

Which transformations and joins get you to one tidy table?

5

Name one quality test you would add

6

Who should not see the raw hotel data?

HOW THIS WORKS

10 min

Work in pairs. Sketch the pipeline on paper as boxes and arrows, the same way as the DAG slide.

We will then look at a few designs together and see where they differ.

Before next class

1

Run the extract

Pull 30 days of USD/CAD from the Bank of Canada Valet API in your Codespace, and save it to data/raw/ as Parquet.

2

Make something tidy

Take one wide StatCan table and reshape it to long format with melt().

3

Try DuckDB

Query your Parquet file with one SQL GROUP BY, for example the average rate by week.

4

Optional: read Wickham (2014)

"Tidy data", sections 1 to 3. It is free online, and short.



STUCK?

Office hours are on Tuesdays, 1:00 to 1:50, in Plaza 429.

Or email imedovikov@brocku.ca, and include the error message you got.

Questions?

```
python · wrap_up.py  
  
>>> stack = ["sources", "storage", "transform",  
...         "orchestrate", "analyze"]  
>>> office_hours = "Tuesdays 1:00, Plaza 429"
```

Ivan Medovikov · imedovikov@brocku.ca

ECON 2P36 · Python for Data and Analytics · Brock University